

Technical Challenge: The Harness

Tech Lead, Agentic Systems | InCommodities North America | v5

Objective

You are inheriting a system that another engineer built the way most systems get built today: with an LLM assisting, but with a person writing, reading, and testing the code. It ingests pipeline electronic bulletin board notices, extracts fields, scores trading relevance, and emits signals. It works, mostly.

This is not a code review. The job is to build the harness that would make reading its code unnecessary: the layer between a specification and a running system that lets an engineer's attention live in the spec and the evidence, and prove the harness works by finding what is wrong with the inherited system from evidence alone.

AI Tooling

Use whatever you use in practice. In the follow-up conversation, we will ask about the choices in your spec, the gates in your harness, and the places you looked at code anyway, so build things you can explain. Running the inherited system needs an API key of your own; one pass over the labeled set is about forty model calls.

Scenario

Energy traders at InCommodities monitor pipeline electronic bulletin boards (EBBs) for notices that signal disruptions to gas flow. A force majeure on a Louisiana pipeline connected to Henry Hub or LNG export terminals can move basis prices within minutes. Traders need fast awareness, and they need to trust what the system tells them. A false signal that triggers a position adjustment is worse than no signal at all.

The repository you inherit is adapted from work by an engineer now on the team. It comes with a set of labeled notices and documentation the engineer wrote. Assume it is the first of many AI components the team ships this year, and that the engineer will read your spec and use your harness. It is at <https://github.com/dan-incommodities/signal-ext-challenge>. The hiring contact will send you read access; clone it into a private repository of your own and work there.

Structure

Four parts. The spec and the harness are where we spend most of our attention. The last part is a conversation with the team.

Phase 1: The Specification

Write the specification you would hand to your harness for this system: what it does, the contracts it honors with the systems it reads from and writes to, the behaviors that must hold, and the failure modes it must refuse. Aim for something precise enough that an agent could build from it and a harness could derive its checks from it, without a person translating in between.

Specify what the running system must remember across scrapes: earlier notices, revisions, and what has already been signaled. Describe the required behavior when a prior notice is missing, arrives late, or is reprocessed, and what history must remain available to explain a past decision.

This is the document the engineer will maintain instead of the code, so write it for them. The question we will be asking ourselves, and would like you to ask yourself: could the engineer who built this system own this document and never need to open a source file again?

Phase 2: The Harness

Write harness.md. It answers one question: what does the engineer look at instead of code?

Walk us through how tests, contract checks, and evaluations come out of the spec rather than being written by hand. What a build has to pass before it is allowed to exist. What the evidence surface is, the thing a person reads to decide a component can be trusted, and what it must never hide. How a failure finds its way back to the spec instead of to a code patch. What happens when the spec changes, and what happens when the model behind the system changes. And what the engineer's first day looks like working this way, because you are designing their job, not only a pipeline.

Explain how context survives beyond the session that built the system. What does a fresh coding agent need to recover about requirements, domain assumptions, design decisions, and known failures? Show how the harness selects and loads that context, how it stays current, and how conflicting or superseded guidance is handled. When a change or failure teaches you something, explain what is retained, who approves changes to the requirements, and how the next session receives the updated context.

For each gate and derived check, tell us why it is there, what it costs, and what you considered instead. Three gates with teeth are worth more to us than twelve that sound thorough.

Phase 3: Run It

Pick at least one component of the inherited system and rebuild it from your specification, through your harness, with whatever agent tooling you like, without reading the generated code.

Then put the harness to work. The inherited system has defects. See what your harness surfaces from evidence alone: eval results, contract checks, invariants, negative controls. Tell us what it found and how much autonomy you would give this system today under your own standards. The repository also includes an unlabeled set of recent NGPL notices, HTML and PDF. Running your harness against it is optional; if you do, we would like to hear what the evidence surface says about notices nobody has labeled.

Next, start a fresh agent session with no access to the original conversation and make one small maintenance change to your rebuilt component: a changed source format or a newly discovered failure is enough. Use the same spec and evidence approach. Show how the agent recovers the relevant requirements and previous decisions, makes the change, checks existing behavior, and preserves what it learned for the next session. Record the starting request, the context loaded, any extra explanation you had to provide, and the resulting evidence.

Then the part we read most closely: where did you end up looking at the code anyway, why, and what does each of those moments tell you about what the harness is still missing? Nobody gets to zero code reads in ten hours, and we are not expecting you to. A candid map of the harness's edges, drawn by the person who built it, is worth more to us than a longer list of findings.

Phase 4: The Conversation

We will spend a couple of hours together on what you built. The first part is a conversation about your choices: the spec, the gates, the evidence surface, how context carried into the fresh session, and especially the places where you looked at code anyway, because that is where the interesting design problems live. Expect us to push on those, and to share where we have landed ourselves. The second part is a working session. One of us will take the role of an engineer on the team who does not yet trust the harness and wants to keep reading the code, with real reasons. This is a conversation you would be having in your first month here, and we would rather have it with you now. Hold your position, adjust it, or drop it, and tell us which and why. There is no answer we are waiting for. We want to see how you bring someone along.

Time Expectation

Eight to ten hours. A quarter on the spec, a quarter on the harness design, half on running it, including the fresh-session maintenance exercise. Keep that exercise to one small change on the same component. Send what you have at the end of the window and tell us what you would have done with more time. A partial harness with a candid edge map tells us more than a complete one that claims more than it shows.

Deliverables

- Private GitHub repository shared with the hiring contact (the GitHub account that sent you the invitation), containing spec.md, harness.md, the harness code, the rebuilt component, and the coverage report, including where you looked at the code anyway and the record of the fresh-session maintenance exercise. Include the context artifacts needed to repeat that exercise.
- README that explains how to run the harness against the inherited system and start a fresh agent session with the required context

What We Pay Attention To

Whether the spec could replace the code as the thing an engineer owns. Whether the harness derives its checks from the spec or hand-writes them. How much the harness finds from evidence, and how openly the gap is described. Whether a fresh session can recover the context needed to maintain the system, and whether new learning remains available to the next session. Whether your sense of which findings matter reflects real risk on a trading desk. And how you work through disagreement with someone who has to live with your standards.

We are not scoring code craftsmanship, and we will spend more of our time together on what did not work than on what did, because that is where we learn how you think.

Why This Challenge

This is the work. The team is moving from engineers who write and review every line to engineers who define the boundaries, contracts, and validation that make agent-produced work safe to depend on, and the person in this role helps lead that move across a growing team. The challenge is a compressed version of the first months: take one real system, build the layer that lets an engineer trust and maintain it across sessions without reading its code, and bring that engineer along. If that is the problem you want to spend the next few years on, we would like to see how you approach it.

We do not expect you to be a gas markets expert. The trading signal definition below tells you what matters. Where domain questions block you, document the assumption you made and proceed. We will discuss those assumptions in the interview.

Trading Signal Definition

A trading signal is a pipeline notice indicating a meaningful disruption likely to affect gas flow volumes or regional prices. Strong signals typically have:

- Critical or unplanned events (force majeure, urgent disruptions)
- Planned outages with material impact on flow
- Identifiable pipeline segment
- Measurable curtailment volume in mmbtu or Dth
- Geographic relevance, particularly Louisiana pipelines connected to Henry Hub or LNG export terminals

Routine notices, administrative updates, and minor capacity changes are not signals. Notices that supersede earlier ones for the same event should not generate duplicate signals.

Pipeline EBB Sources

The inherited system covers NGPL, and the notices in the repository were scraped from its board. The other boards are listed for context; nothing in this challenge requires scraping.

- NGPL: <https://pipeline2.kindermorgan.com/Notices/Notices.aspx?type=C&code=NGPL>
- GulfSouth: <https://infopost.bwpipelines.com/?tspid=1>
- Creole Trail: <https://lngconnection.cheniere.com/#/home>
- Columbia Gulf: <https://ebb.tceconnects.com/infopost/Default.aspx?nomobile=1>
- ANR: <https://ebb.anrpl.com/>
- Sabine Pipeline: <https://www.gasnom.com/ip/SABINE/>
- TETCO: <https://infopost.enbridge.com/infopost/TEHome.asp?Pipe=TE>